

CYS 4151 Ethical Hacking Lab

Network Configuration & Build Guide

SecureTech Solutions Ltd. — Enterprise Security Assessment

Scope. Build and document an isolated VirtualBox + pfSense laboratory that reproduces SecureTech Solutions Ltd.'s network so that a Red Team and a Blue Team can carry out the authorized assessment required by *CYS 4151 Final Examination, Setting A, Summer 2026* (Dr. Nkandeu Pascal). All testing is confined to 192.168.50.0/24.

Lab Team

PC1 Marshall

PC2 Fahdil

PC3 Rayan

PC4 Bruno

PC5 Amina

Contents

1	Introduction & Scenario Recap	2
2	Team & Role Assignment	2
3	Network Topology	2
4	Design-Level Strategy	3
5	PC1 — Marshall: pfSense Gateway + Suricata IDS (Blue Team)	5
5.1	Install VirtualBox and create the pfSense VM	5
5.2	Network adapters for pfSense	5
5.3	Initial pfSense console staging	5
5.4	pfSense web GUI initial setup wizard	6
5.5	Suricata-in-Kali IDS VM (second VM on PC1)	6
6	PC2 — Fahdil: Windows Workstation (Blue Team)	8
7	PC3 — Rayan: Server & Vulnerable Targets (Infrastructure, then Red Team)	9
7.1	Host server VM	9
7.2	Nested target: DVWA / OWASP Juice Shop	9
7.3	Nested target: Metasploitable2/3	10
8	PC4 — Bruno: Kali Linux Attacker Platform (Red Team)	12
9	PC5 — Amina: OpenVAS / Greenbone Vulnerability Scanner (Blue Team)	13
10	pfSense Access List & Firewall Rule Configuration	15
10.1	Define aliases	15
10.2	Ordered LAN interface rules	15
10.3	Verify the ruleset	17
11	End-to-End Connectivity Verification	18
12	Mandatory Evaluation Data Overlay (Dr. Nkandeu Pascal)	18

1 Introduction & Scenario Recap

SecureTech Solutions Ltd. is a 120-employee software and digital services company that reported unauthorized login attempts, weak/reused passwords, suspicious web server activity, possible customer data exposure, unpatched systems, and no centralized monitoring. Our team was authorized, in writing, to simulate both a **Red Team** (offensive) and a **Blue Team** (defensive) inside a contained laboratory only — never against production systems.

This document covers **Task 1 (Laboratory Design & Documentation)**: the physical and virtual network we built, every VM from first install to a verified static IP, the real problems we hit while bringing the network up, and the pfSense access-control configuration that enforces the lab's security boundary. The companion *Post-Configuration Execution Guide* picks up from here and walks Tasks 2–13.

Authorized Scope

All offensive and defensive activity described in both documents is confined to the isolated lab segment 192.168.50.0/24. Nothing here is ever pointed at a host outside this range.

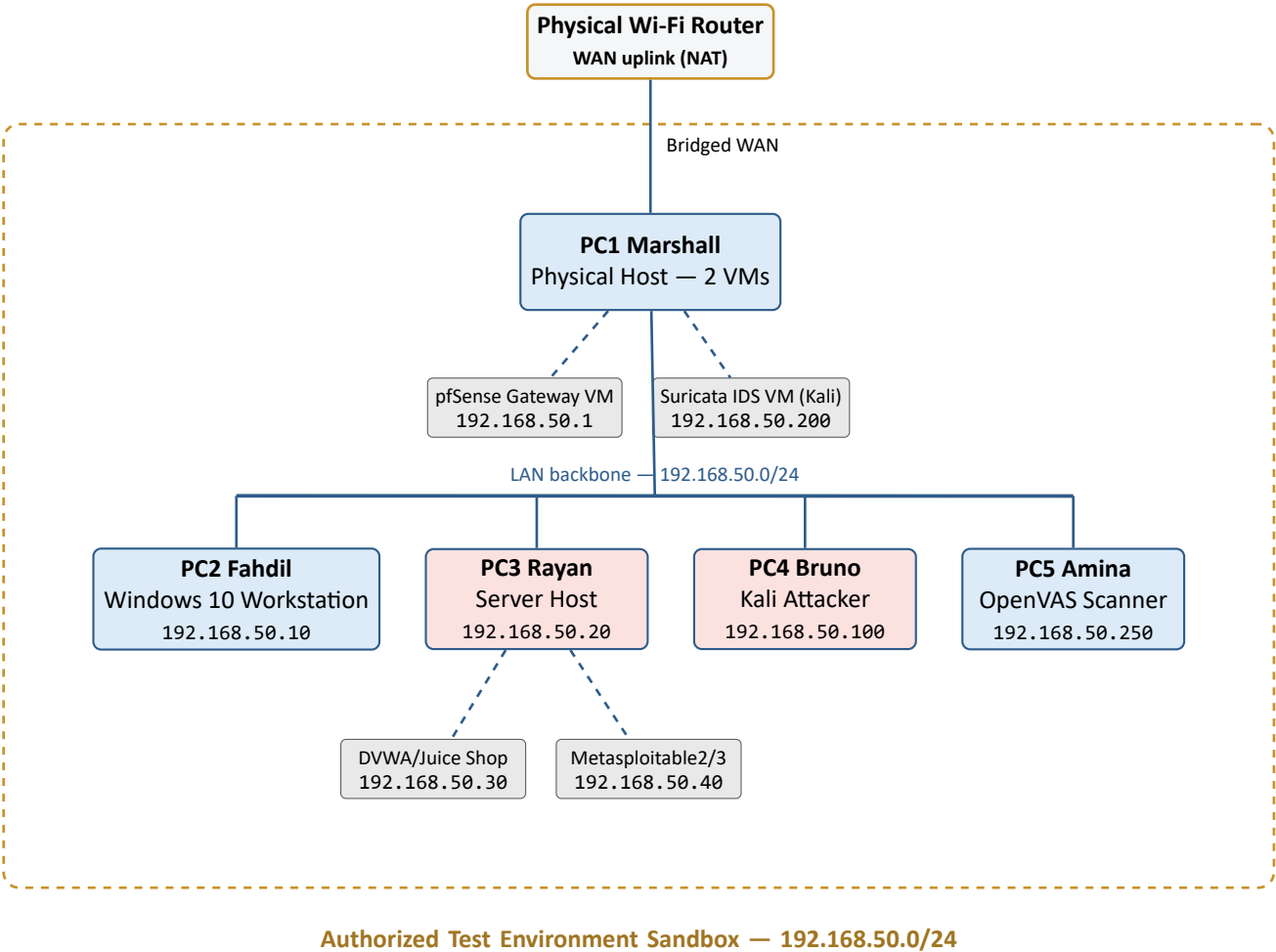
2 Team & Role Assignment

Five physical host machines were used, one per student, each running one or more VirtualBox guest VMs. Team colour follows the badge used throughout both documents.

PC	Team	Lab IP(s)	Primary Responsibility
PC1 Marshall	Blue	192.168.50.1	pfSense firewall/router (gateway for the whole lab) + a Kali VM running Suricata as an inline-monitoring IDS sensor.
PC2 Fahdil	Blue	192.168.50.10	Windows 10 workstation representing the “employee computer” asset; Blue Team hardens and audits it (password policy, patching).
PC3 Rayan	Red*	192.168.50.20 (+.30, +.40 nested)	Builds and hosts the shared Server + DVWA/Juice Shop + Metasploitable2/3 targets; once the network is live, operates as Red Team against the wider lab.
PC4 Bruno	Red	192.168.50.100	Kali Linux — primary offensive attack platform (recon, enumeration, exploitation).
PC5 Amina	Blue	192.168.50.250	OpenVAS/Greenbone vulnerability scanner — automated assessment and CVSS triage.

*Rayan configures PC3 as shared infrastructure during the build phase (Section 7); once connectivity is verified he switches hats and works exclusively as a Red Team member for Tasks 2 onward.

3 Network Topology



Blue Team (Defense) Red Team (Offense) Vulnerable/Nested Target VM External/Physical Infrastructure

Figure 1: Corrected lab topology: PC1 (Marshall) and PC5 (Amina) anchor the Blue Team defensive stack (firewall, IDS, and vulnerability scanning); PC2 (Fahdil) is the Blue Team’s hardened workstation asset; PC3 (Rayan) hosts the shared vulnerable targets and transitions to Red Team once the network goes live; PC4 (Bruno) is the Red Team’s primary attack platform.

4 Design-Level Strategy

Red Team — Network-Level Intent

Rayan and Bruno treat 192.168.50.0/24 as the target range. Their plan is to (1) sweep the whole sub-net from Bruno’s Kali box (PC4) to build an asset map, (2) fingerprint the server/web/target stack hosted on Rayan’s PC3, (3) look for a foothold on the Metasploitable/DVWA targets first since they are intentionally weak, then (4) pivot from that foothold toward Fahdil’s workstation (PC2) to simulate lateral movement toward an “employee” asset, all while assuming pfSense (PC1) and the OpenVAS/Suricata nodes are watching — i.e. they expect to be logged and plan noise-reduction (timing, targeted scans) accordingly.

Blue Team — Network-Level Intent

Marshall, Fahdil and Amina assume the Red Team will enter through the weakest advertised services (the DVWA/Metasploitable targets on PC3). Their plan is to (1) make pfSense (PC1) the single choke-point all inter-host traffic must cross, with a default-deny LAN ruleset and only the narrow exceptions defined in Section 10; (2) run Suricata in the Kali VM on PC1 in promiscuous IDS mode against the

LAN backbone so every host-to-host flow is visible for detection, not just WAN traffic; (3) run OpenVAS from PC5 proactively against the server/target stack so known vulnerabilities are documented *before* Red Team confirms them by exploiting them; and (4) keep Fahdil's workstation (PC2) patched and its credential policy audited so it is not the easy pivot target Red Team is hoping for.

5 PC1 — Marshall: pfSense Gateway + Suricata IDS (Blue Team)

PC1 is the centralized firewall, router and traffic-inspection point for the entire lab. It runs *two* guest VMs under one VirtualBox hypervisor: a pfSense VM (LAN gateway/firewall) and a Kali Linux VM dedicated to running Suricata as a passive IDS sensor.

5.1 Install VirtualBox and create the pfSense VM

1. Install Oracle VirtualBox (and the matching Extension Pack) on the PC1 host machine.
2. Download the current pfSense CE installer ISO from the official pfSense site.
3. In VirtualBox: **New** → Name pfSense-GW, Type BSD, Version FreeBSD (64-bit).
4. Allocate **2048 MB RAM**, **2 vCPUs**, and a **20 GB** dynamically allocated VDI disk (pfSense itself is light; this leaves headroom for packages and logs).
5. Settings → Storage: attach the pfSense ISO to the virtual optical drive.
6. Settings → System → Processor: enable **PAE/NX**.

5.2 Network adapters for pfSense

pfSense needs two NICs: one for the uplink (WAN) and one for the lab LAN.

1. Settings → Network → **Adapter 1 (WAN)**: Attached to **NAT**. This lets pfSense reach the internet (package/plugin downloads, patches) without exposing the lab.
2. Settings → Network → **Adapter 2 (LAN)**: Attached to **Bridged Adapter**, Name = the physical host's active Wi-Fi adapter. Expand **Advanced** and set **Promiscuous Mode: Allow All**.

Host-Only Adapter attempt failed

Our first attempt used a **Host-Only Adapter** for the LAN leg on every PC, assuming it would behave like a normal lab switch. It did not work: a VirtualBox Host-Only network is a virtual switch that exists *only inside one physical hypervisor* — it never leaves that physical machine's NIC. Because our five students are on five separate physical PCs connected only over Wi-Fi, each PC's VMs ended up on their own isolated Host-Only segment with no path to the other four PCs: pings between VMs on different physical machines timed out, DHCP requests to pfSense never arrived, and pfSense's LAN interface only ever saw its own host's loopback traffic. **Fix:** switch every LAN-side adapter (pfSense and all four clients) to **Bridged Adapter** bound to each machine's physical Wi-Fi NIC, with **Promiscuous Mode: Allow All**. Bridging makes every VM appear as its own device directly on the physical Wi-Fi segment, so all five physical PCs' VMs can finally see one another.

5.3 Initial pfSense console staging

1. Power on the pfSense VM and let the installer finish; reboot into the console menu.
2. At the console menu, choose **2) Set interface IP address**.
3. Select the LAN interface, assign the static address `192.168.50.1`.
4. Enter the subnet bitmask: `24`.
5. When prompted, enable the DHCP server on LAN (optional — we assign static addresses to every client instead, see Sections 6–9).

Expected Outcome

The console menu reports the LAN interface as `192.168.50.1/24, UP`. From any host already bridged onto the LAN segment:

```
# confirm the gateway answers on the lab subnet
```

```
ping -c 4 192.168.50.1
```

returns 4 replies with low, stable latency.

5.4 pfSense web GUI initial setup wizard

1. From a bridged client, browse to `https://192.168.50.1` and accept the self-signed certificate warning.
2. Log in with the default credentials (admin / pfsense) and immediately change the admin password under the setup wizard.
3. Set the hostname (pfsense-gw), domain, and DNS servers.
4. Confirm the WAN interface shows an address obtained via NAT/DHCP from the VirtualBox NAT engine, and the LAN interface shows `192.168.50.1/24`.
5. Finish the wizard and reload the GUI.

5.5 Suricata-in-Kali IDS VM (second VM on PC1)

A second guest VM on the same PC1 hypervisor runs Kali Linux purely as a Suricata sensor — it is never used to attack, only to listen.

1. VirtualBox **New** → Name PC1-Suricata-IDS, Type Linux, Version Debian (64-bit). Allocate 4096 MB RAM, 2 vCPUs, 40 GB disk.
2. Attach the Kali Linux installer ISO and complete a standard installation.
3. Settings → Network → Adapter 1: **Bridged Adapter** on the same physical Wi-Fi NIC, **Promiscuous Mode: Allow All** (required so this VM can passively see LAN traffic that is not addressed to it).
4. Assign it a static management address, `192.168.50.200/24`, gateway `192.168.50.1`.

Suricata installation & ruleset update

```
# update package lists and install Suricata
sudo apt update && sudo apt install -y suricata jq

# confirm the interface that should see LAN traffic
ip -brief addr show

# point Suricata at the lab subnet (HOME_NET line is indented under
# vars: address-groups: in suricata.yaml, so the pattern must allow
# leading whitespace or the substitution silently matches nothing)
sudo sed -i 's/^( *HOME_NET:).*$/\1 "[192.168.50.0/24]"/' \
    /etc/suricata/suricata.yaml

# confirm the substitution actually took effect
grep "HOME_NET:" /etc/suricata/suricata.yaml

# pull the current ET-open ruleset (a fresh install ships almost no rules)
sudo suricata-update
```

Why suricata-update is required

A freshly installed Suricata only ships a handful of example rules. Without running `suricata-update` at least once, most ET-open signatures referenced later in this guide and in Task 11 of the Execution Guide (scan, exploit, and web-attack categories) simply do not exist yet, so nothing would alert regardless of how the sensor is wired up.

Custom local rule: detect a port-scan threshold

Suricata does not flag a plain SYN scan out of the box the way a scanner-detection engine would — this has to be added as a local rule using a threshold clause, which counts matching packets per source over a time window.

```
sudo nano /etc/suricata/rules/local.rules
```

Contents of local.rules

```
alert tcp any any -> $HOME_NET any (msg:"LOCAL Potential TCP Port Scan"; \
  flags:S; threshold:type threshold, track by_src, count 20, seconds 10; \
  classtype:attempted-recon; sid:1000001; rev:1;)
```

Confirm local.rules is loaded, then start Suricata

```
# the default rule-files list may not reference local.rules yet --
# add it right after the "rule-files:" line only if it's missing
grep -qxF ' - local.rules' /etc/suricata/suricata.yaml || \
  sudo sed -i '/rule-files:/a\ - local.rules' /etc/suricata/suricata.yaml

# validate the configuration and new rule before going live
sudo suricata -T -c /etc/suricata/suricata.yaml -v

# enable and start Suricata in IDS (listen-only) mode on the bridged interface
sudo systemctl enable --now suricata

# tail live alerts as JSON, filtered to readable fields
sudo tail -f /var/log/suricata/eve.json | jq 'select(.event_type=="alert")'
```

Expected Outcome

`systemctl status suricata` reports active (running). The `eve.json` tail stays idle until traffic matching a signature crosses the bridged interface; once Red Team traffic starts in the companion Execution Guide, alert JSON objects appear here in real time — this is the evidence base for Task 11 (Blue Team Investigation).

Why a Kali VM for Suricata instead of pfSense's own package

pfSense ships a Suricata package that can run inline on its own interfaces, but we deliberately run Suricata on a separate Kali VM in *promiscuous, out-of-band* mode instead. This keeps the firewall VM (PC1's pfSense guest) lean and dedicated purely to routing/filtering, while the IDS sensor can be rebuilt, tuned, or have its ruleset updated without any risk of taking down the lab's only gateway.

6 PC2 — Fahdil: Windows Workstation (Blue Team)

PC2 represents the “employee computer” asset named in the exam brief. Fahdil, as Blue Team, is responsible for keeping it patched and auditing its credential policy (feeds Task 8 and Task 12 in the Execution Guide).

1. VirtualBox **New** → Name PC2-Workstation, Type Microsoft Windows, Version Windows 10 (64-bit). Allocate 4096 MB RAM, 2 vCPUs, 60 GB disk.
2. Attach a Windows 10 installer ISO and complete a standard installation with a local employee-style account (no Microsoft account, to mirror a realistic corporate workstation).
3. Install the VirtualBox Guest Additions (Devices → Insert Guest Additions CD image) for better display/clipboard integration.
4. Settings → Network → Adapter 1: **Bridged Adapter** on the host’s physical Wi-Fi NIC, **Promiscuous Mode: Allow All**.
5. Inside Windows, open **Control Panel** → Network and Sharing Center → Change adapter settings → IPv4 Properties and set a static configuration: IP 192.168.50.10, Mask 255.255.255.0, Gateway 192.168.50.1, DNS 192.168.50.1 (forwarded by pfSense).

Verify Wi-Fi MAC and connectivity (PowerShell / Command Prompt)

```
:: confirm the physical Wi-Fi adapter MAC (avoids duplicate-MAC conflicts)
getmac /v

:: confirm the static address took effect
ipconfig /all

:: confirm the pfSense gateway answers
ping -n 4 192.168.50.1
```

Expected Outcome

ipconfig /all shows IPv4 Address 192.168.50.10, Subnet Mask 255.255.255.0, Default Gateway 192.168.50.1. The ping to the gateway returns 4 replies.

Difficulty: duplicate MAC warning during early bridging tests

While still experimenting with Host-Only networking (see Section 5), we briefly cloned a VM instead of reinstalling, which duplicated a MAC address across two guests. Once both were bridged onto the same physical Wi-Fi segment this caused ARP cache thrashing and intermittent dropped pings. **Fix:** in VirtualBox, Settings → Network → Advanced → **Generate new MAC address** on the cloned adapter before powering it on, and never clone an already-bridged VM without doing this.

7 PC3 — Rayan: Server & Vulnerable Targets (Infrastructure, then Red Team)

PC3 hosts the shared application/file server and the two intentionally vulnerable targets the whole exam is built around. Rayan configures and owns this machine as *shared lab infrastructure*; once connectivity to every other PC is verified he puts on the Red Team hat and attacks these same targets (and others) for Tasks 2 onward in the Execution Guide.

7.1 Host server VM

1. VirtualBox **New** → Name PC3-Server, Type Linux, Version Ubuntu (64-bit) (Ubuntu Server 22.04 LTS used here). Allocate 4096 MB RAM, 2 vCPUs, 60 GB disk.
2. Install Ubuntu Server with OpenSSH enabled during setup.
3. Settings → Network → Adapter 1: **Bridged Adapter**, Promiscuous Mode **Allow All**.
4. Configure a static address via netplan.

Static IP via netplan (Ubuntu Server)

```
# edit the netplan config (filename may differ, e.g. 00-installer-config.yaml)
sudo nano /etc/netplan/00-installer-config.yaml
```

netplan YAML contents

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses: [192.168.50.20/24]
      routes:
        - to: default
          via: 192.168.50.1
      nameservers:
        addresses: [192.168.50.1]
```

Apply and verify

```
# apply the new network configuration
sudo netplan apply

# confirm the address took effect
ip -brief addr show

# confirm the gateway answers
ping -c 4 192.168.50.1
```

Expected Outcome

ip addr lists 192.168.50.20/24 on the bridged interface; the ping to 192.168.50.1 returns 4 replies.

7.2 Nested target: DVWA / OWASP Juice Shop

1. Install Docker on the PC3 host so the vulnerable web app runs in an isolated container reachable at its own lab IP via a macvlan/bridge network, or alternatively install it as a second nested VirtualBox VM — either satisfies the exam’s “Vulnerable Web Application” requirement. The container route is shown below.

Run DVWA in Docker, given its own lab IP via macvlan

```
# install docker engine
sudo apt update && sudo apt install -y docker.io

# create a macvlan network so the container is a first-class LAN host,
# not just a port forwarded through the PC3 host's own IP
sudo docker network create -d macvlan \
  --subnet=192.168.50.0/24 --gateway=192.168.50.1 \
  -o parent=enp0s3 lanmacvlan

# run DVWA attached to that network with its own static address
sudo docker run -d --name dvwa --network lanmacvlan \
  --ip 192.168.50.30 vulnerables/web-dvwa

# confirm the container is up
sudo docker ps
```

Expected Outcome

docker ps shows the dvwa container Up. From any other bridged host:

```
curl -I http://192.168.50.30
```

returns an HTTP/1.1 200 OK (or a redirect to the DVWA login page).

7.3 Nested target: Metasploitable2/3

1. VirtualBox **New** → Name PC3-Metasploitable, Type Linux, Version Other Linux (64-bit). Allocate 1024 MB RAM, 1 vCPU, the prebuilt Metasploitable2 VMDK (8 GB) attached directly — no fresh install needed, it ships as a ready-made vulnerable image.
2. Settings → Network → Adapter 1: **Bridged Adapter**, Promiscuous Mode **Allow All**.
3. Log in with the default credentials (msfadmin/msfadmin) and set a static address.

Static IP on Metasploitable2 (legacy Debian network config)

```
# edit the legacy interfaces file (Metasploitable2 predates netplan)
sudo nano /etc/network/interfaces
```

/etc/network/interfaces contents

```
auto eth0
iface eth0 inet static
    address 192.168.50.40
    netmask 255.255.255.0
    gateway 192.168.50.1
```

Apply and verify

```
sudo /etc/init.d/networking restart
ifconfig eth0
ping -c 4 192.168.50.1
```

Expected Outcome

ifconfig eth0 shows inet addr:192.168.50.40; the ping to the gateway succeeds.

Note from Rayan's dual role

Because Rayan both builds these targets and later attacks them as Red Team, he keeps a clean baseline snapshot of each nested VM (VirtualBox **Machine** → **Take Snapshot**) immediately after this section, *before* any Task 2+ offensive activity. This lets the team reset a target to its known-vulnerable state between exploitation attempts without rebuilding it from scratch.

8 PC4 — Bruno: Kali Linux Attacker Platform (Red Team)

PC4 is the Red Team's primary offensive workstation for Tasks 2–10 in the Execution Guide (reconnaissance through post-exploitation).

1. VirtualBox **New** → Name PC4-Kali-Attacker, Type Linux, Version Debian (64-bit). Allocate 4096 MB RAM, 2 vCPUs, 80 GB disk (wordlists, scan output, and loot accumulate quickly).
2. Attach the official Kali Linux installer ISO and complete a standard installation with the full tool meta-package.
3. Settings → Network → Adapter 1: **Bridged Adapter** on the host's physical Wi-Fi NIC, **Promiscuous Mode: Allow All**.
4. Assign a static address.

Static IP via netplan + first tool verification

```
# netplan config for Kali's default interface (name may vary, check with: ip link)
sudo nano /etc/netplan/01-network-manager-all.yaml
```

netplan YAML contents

```
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: no
      addresses: [192.168.50.100/24]
      routes:
        - to: default
          via: 192.168.50.1
      nameservers:
        addresses: [192.168.50.1]
```

Apply, update, and confirm the toolkit is ready

```
# apply networking and bring the tool database up to date
sudo netplan apply
sudo apt update && sudo apt full-upgrade -y

# confirm addressing and reachability
ip -brief addr show
ping -c 4 192.168.50.1

# sanity-check the core tools used throughout the Execution Guide are present
nmap --version
msfconsole --version
hydra -h | head -n 1
searchsploit -h | head -n 1
```

Expected Outcome

ip addr shows 192.168.50.100/24; the gateway ping succeeds; each tool prints a version/usage banner instead of "command not found".

9 PC5 — Amina: OpenVAS / Greenbone Vulnerability Scanner (Blue Team)

PC5 runs the automated vulnerability assessment platform used in Task 5. This is also where the lab hit its most serious build problem.

1. VirtualBox **New** → Name PC5-OpenVAS, Type Linux, Version Debian (64-bit). Allocate **8192 MB RAM**, 4 vCPUs.
2. Allocate a **80 GB** (dynamically allocated) virtual disk — see the difficulty note below for why this number matters.
3. Install Kali Linux (ships a one-command GVM/OpenVAS setup) or a minimal Debian with the Greenbone Community Edition packages.
4. Settings → Network → Adapter 1: **Bridged Adapter**, Promiscuous Mode **Allow All**. Assign a static address 192.168.50.250/24, gateway 192.168.50.1.

Install and initialize GVM/OpenVAS

```
# install the Greenbone Vulnerability Management suite
sudo apt update && sudo apt install -y gvm

# one-time setup: builds the DB, syncs NVT/SCAP/CERT feeds, creates admin user
sudo gvm-setup

# verify the install is healthy before first use
sudo gvm-check-setup
```

Difficulty: gvm-setup / feed sync repeatedly failed

Our first VM for PC5 was built with the same 20–40 GB disk we used for the other hosts. `gvm-setup` and `gvm-check-setup` failed repeatedly with feed synchronization errors and PostgreSQL write failures. The root cause was **insufficient virtual disk space**, not insufficient video memory: the combined NVT, SCAP and CERT feed data plus the PostgreSQL vulnerability database that GVM builds on first run require well over 40 GB once fully synced, and our undersized disk filled up mid-sync, corrupting the partial feed state on every retry.

Fix:

1. Power off the VM. In VirtualBox, use **File** → **Virtual Media Manager** to increase the VDI's logical size to 80 GB (or clone to a larger disk).
2. Boot into the guest and grow the filesystem to use the new space.

```
# identify the root partition/volume
lsblk

# grow the root partition to fill the new disk size (distro-dependent)
sudo growpart /dev/sda 1
sudo resize2fs /dev/sda1

# re-run the health check until it reports no errors
sudo gvm-check-setup

# re-trigger feed synchronization if any feed reported stale/incomplete
# (--type values are lowercase: nvt, scap, cert, gvm-data)
sudo greenbone-feed-sync --type nvt
sudo greenbone-feed-sync --type scap
sudo greenbone-feed-sync --type cert
sudo greenbone-feed-sync --type gvm-data
```

Start services and verify via the web UI

```
# start the GVM stack (postgres, gvmd, ospd-openvas, gsad)
sudo gvm-start
```

```
# confirm the web UI port is listening
ss -tlnp | grep 9392
```

Expected Outcome

gvm-check-setup ends with "It seems like your GVM-21... install is OK." Browsing to <https://192.168.50.250:9392> from any bridged host shows the Greenbone Security Assistant login page, and logging in with the admin credentials generated during gvm-setup succeeds.

10 pfSense Access List & Firewall Rule Configuration

This section configures PC1's pfSense gateway so it enforces, not just routes, the lab boundary. We follow a default-deny model: nothing is permitted on the LAN interface unless it is explicitly listed below.

10.1 Define aliases

Aliases let the ruleset reference groups of hosts by name instead of raw IPs, so rules stay readable and are easy to re-point if an address changes.

1. **Firewall** → **Aliases** → **Add**.
2. Create `BlueTeam_Hosts` (type: Host(s)) with members `192.168.50.10` (Fahdil), `192.168.50.200` (Suricata sensor), `192.168.50.250` (Amina/OpenVAS).
3. Create `RedTeam_Hosts` (type: Host(s)) with members `192.168.50.20` (Rayan), `192.168.50.100` (Bruno).
4. `Target_Hosts` — type Host(s); members `192.168.50.20`, `192.168.50.30`, `192.168.50.40`.
5. Create `Mgmt_Ports` (type: Port(s)) with `443` (pfSense GUI), `9392` (GVM web UI), `22` (SSH).
6. **Save**, then **Apply Changes**.

10.2 Ordered LAN interface rules

pfSense evaluates LAN rules top-to-bottom, first match wins, so order matters. We keep an explicit anti-lockout allowance first, then narrow allows, then a logged catch-all deny at the bottom (defence in depth even though the LAN tab's own implicit deny would otherwise apply).

#	Protocol	Source	Destination	Port	Action	Purpose
1	TCP	any	This Firewall	443	Pass	Anti-lockout: always allow GUI access to pfSense itself.
2	ICMP	LAN net	LAN net	–	Pass	Allow ping between all lab hosts (needed for every connectivity check in this guide).
3	TCP/UDP	BlueTeam_Hosts	LAN net	any	Pass	Blue Team management/monitoring traffic (Suricata, OpenVAS scans) reaches every target.
4	TCP	RedTeam_Hosts	Target_Hosts	21, 22, 80, 443, 445, 3306	Pass	Red Team may only reach the declared target service ports, not arbitrary LAN hosts.
5	TCP	RedTeam_Hosts	!Target_Hosts	any	Block & log	Explicitly blocks Red Team from reaching anything outside the declared target set (e.g. Fahdil's workstation) by default.
6	any	LAN net	WAN net	any	Pass	Permit outbound internet for updates/feeds (NAT on WAN handles translation).
7	any	any	any	any	Block & log	Default-deny catch-all; anything not explicitly matched above is dropped and logged.

1. **Firewall** → **Rules** → **LAN** → **Add** for each row above, matching Source/Destination/Port exactly as listed.
2. For rules 5 and 7, tick **Log packets that are handled by this rule** so blocked attempts appear in **Status** → **System Logs** → **Firewall** (this feeds Task 11's log-based detection evidence).
3. Use the drag handles on the Rules page to confirm the final order matches the table (1 at the top, 7

at the bottom).

4. Apply Changes.

Why rule 4 constrains Red Team by port, not just by host

Rule 4 only passes the specific service ports the exam's vulnerable targets actually expose (FTP/SSH/HTTP/HTTPS/SMB/MySQL). This mirrors a real segmented network: the firewall does not need to know *which* vulnerability Red Team will use, only that their traffic should never reach anything beyond the declared target surface.

10.3 Verify the ruleset

From Bruno's Kali (PC4) — confirm allowed vs. blocked paths

```
# allowed: reaches a declared target service (DVWA on port 80)
curl -I http://192.168.50.30

# blocked by rule 5: Red Team host reaching a non-target LAN host directly
nc -zv -w 3 192.168.50.10 445
```

Expected Outcome

The `curl` to 192.168.50.30 returns HTTP headers (allowed path). The `nc` attempt to Fahdil's workstation (192.168.50.10) times out or is actively refused (blocked path). In the pfSense GUI, **Status** → **System Logs** → **Firewall** shows a logged block entry for the second attempt, with Bruno's IP as source.

1. **Diagnostics** → **States** on the pfSense GUI to watch live connection state entries as each test above runs — confirms which rule number actually matched.
2. **Status** → **System Logs** → **Firewall** → use the built-in filter to search by source IP and confirm the rule number/action logged matches the table in Section 10.2.

11 End-to-End Connectivity Verification

Run from Bruno's Kali box (PC4) once every PC above is bridged and addressed, to confirm the whole lab is reachable before any exam task begins.

Full connectivity sweep

```
# 1. central pfSense gateway
ping -c 4 192.168.50.1

# 2. Fahdil's workstation (Blue Team asset)
ping -c 4 192.168.50.10

# 3. Rayan's server host
ping -c 4 192.168.50.20

# 4. nested DVWA / Juice Shop target
ping -c 4 192.168.50.30

# 5. nested Metasploitable target
ping -c 4 192.168.50.40

# 6. Suricata IDS sensor (management address, Kali on PC1)
ping -c 4 192.168.50.200

# 7. Amina's OpenVAS scanner (Kali on PC5)
ping -c 4 192.168.50.250
```

Expected Outcome

All seven hosts reply with low, stable latency. If any host fails, re-check (in order): the adapter is **Bridged**, not Host-Only; **Promiscuous Mode** is **Allow All**; the static IP/gateway match this document; and pfSense rule 2 (Section 10) is in place and above the catch-all deny.

Host	IP	Team
PC1 Marshall pfSense GW	192.168.50.1	Blue
PC1 Suricata sensor (Kali)	192.168.50.200	Blue
PC2 Fahdil Workstation	192.168.50.10	Blue
PC3 Rayan Server host	192.168.50.20	Red*
DVWA/Juice Shop (nested)	192.168.50.30	Target
Metasploitable2/3 (nested)	192.168.50.40	Target
PC4 Bruno Kali attacker	192.168.50.100	Red
PC5 Amina OpenVAS scanner (Kali)	192.168.50.250	Blue

12 Mandatory Evaluation Data Overlay (Dr. Nkandeu Pascal)

To secure the 5 marks allocated to Task 1 (Laboratory Design & Documentation), every screen recording deliverable — in this document and the companion Execution Guide — must display:

- Student identification text: full legal name and Student ID visible throughout.
- Environmental context: visible hostname and active user account.
- Temporal tracking: the host system's real-time date and time visible on screen.
- Continuity: unedited, continuous captures with narration for every major milestone.

Scope Reminder

All offensive testing in both documents is confined to this isolated, authorized lab network, 192.168.50.0/24.

Continue to *CYS 4151 Post-Configuration Execution Guide* for Tasks 2–13.